

```

import os
import asyncio
import logging
from datetime import datetime
import httpx
from apscheduler.schedulers.asyncio import AsyncIOScheduler
from telegram import Bot

# --- CONFIG ---
TELEGRAM_TOKEN = os.getenv("TELEGRAM_TOKEN", "8783177635:AAFdsz2X3Myp8mnweuT6gRrv")
CHAT_ID = os.getenv("CHAT_ID", "8507279948")
ODDS_API_KEY = os.getenv("ODDS_API_KEY", "00ba40654da2cc8a63aa56b513d46b18")
SEND_HOUR = 8 # heure d'envoi (UTC)
SEND_MINUTE = 0
MIN_VALUE = 3 # % de valeur minimum pour signaler un match
# ---

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger(__name__)

def implied_prob(odd: float) -> float:
    """Probabilité implicite d'une cote."""
    return 1 / odd if odd > 0 else 0

def value_percent(fair_prob: float, odd: float) -> float:
    """Value = (prob_réelle × cote) - 1, en %."""
    return round((fair_prob * odd - 1) * 100, 1)

def analyze_match(match: dict) -> dict | None:
    """
    Calcule la value bet sur un match.
    On compare la cote d'un bookmaker à la moyenne de tous les bookmakers
    pour estimer la 'vraie' probabilité.
    Retourne None si aucune value intéressante.
    """
    home = match.get("home_team", "")
    away = match.get("away_team", "")
    tournament = match.get("sport_title", "Tennis")
    bookmakers = match.get("bookmakers", [])

    if not bookmakers:

```

```

    return None

# Collecte toutes les cotes disponibles
all_odds_home, all_odds_away = [], []
for bk in bookmakers:
    for market in bk.get("markets", []):
        if market.get("key") != "h2h":
            continue
        outcomes = market.get("outcomes", [])
        for o in outcomes:
            if o["name"] == home:
                all_odds_home.append(o["price"])
            elif o["name"] == away:
                all_odds_away.append(o["price"])

if not all_odds_home or not all_odds_away:
    return None

# Cotes moyennes du marché
avg_home = sum(all_odds_home) / len(all_odds_home)
avg_away = sum(all_odds_away) / len(all_odds_away)

# Marge bookmaker
margin = implied_prob(avg_home) + implied_prob(avg_away)

# Probabilités "justes" (sans marge)
fair_home = implied_prob(avg_home) / margin
fair_away = implied_prob(avg_away) / margin

# Meilleure cote disponible sur le marché
best_home = max(all_odds_home)
best_away = max(all_odds_away)

val_home = value_percent(fair_home, best_home)
val_away = value_percent(fair_away, best_away)

best_val = max(val_home, val_away)
if best_val < MIN_VALUE:
    return None

pick = home if val_home >= val_away else away
pick_odd = best_home if val_home >= val_away else best_away
pick_val = val_home if val_home >= val_away else val_away
pick_prob = round(fair_home * 100 if val_home >= val_away else fair_away * 100)

# Score lisible 0-100
score = min(100, int(50 + pick_val * 2))

```

```

level = "🔥 FORTE VALUE" if pick_val >= 8 else "⚡ BONNE VALUE" if pick_val >

return {
    "home": home, "away": away, "tournament": tournament,
    "pick": pick, "odd": pick_odd,
    "value": pick_val, "prob": pick_prob,
    "score": score, "level": level,
    "avg_home": round(avg_home, 2), "avg_away": round(avg_away, 2),
}

async def fetch_matches() -> list[dict]:
    """Récupère les matchs tennis du jour via The Odds API."""
    url = (
        f"https://api.the-odds-api.com/v4/sports/tennis/odds/"
        f"?apiKey={ODDS_API_KEY}&regions=eu&markets=h2h&oddsFormat=decimal"
    )
    async with httpx.AsyncClient(timeout=15) as client:
        r = await client.get(url)
        r.raise_for_status()
        return r.json()

def build_message(picks: list[dict]) -> str:
    today = datetime.now().strftime("%d/%m/%Y")
    lines = [
        f"🎾 *TENNIS VALUE BETS – {today}*",
        f"—————",
        f"📊 {len(picks)} value bet(s) détecté(s)\n",
    ]

    for i, p in enumerate(picks, 1):
        lines += [
            f"*Match {i}*",
            f"🏆 {p['tournament']}",
            f"👤 {p['home']} vs {p['away']}",
            f"✅ *Pick : {p['pick']}",
            f"💰 Meilleure cote : {p['odd']}",
            f"📈 Value : **{p['value']}%",
            f"🎯 Probabilité réelle estimée : {p['prob']}%",
            f"⚡ Score : {p['score']}/100 – {p['level']}",
            f"—————",
        ]

    lines.append("⚠️ _Outil informatif uniquement. Joue de façon responsable._")
    return "\n".join(lines)

```

```

async def send_picks():
    log.info("Récupération des matchs...")
    try:
        matches = await fetch_matches()
    except Exception as e:
        log.error(f"Erreur fetch matches: {e}")
        return

    if not matches:
        log.info("Aucun match trouvé aujourd'hui.")
        await Bot(TELEGRAM_TOKEN).send_message(
            chat_id=CHAT_ID,
            text="🟡 Aucun match tennis avec cotes disponibles aujourd'hui.",
        )
        return

    picks = []
    for m in matches:
        result = analyze_match(m)
        if result:
            picks.append(result)

    # Trier par value décroissante
    picks.sort(key=lambda x: x["value"], reverse=True)

    if not picks:
        msg = "🟡 Aucune value bet significative détectée aujourd'hui. Journée à
    else:
        msg = build_message(picks)

    bot = Bot(TELEGRAM_TOKEN)
    await bot.send_message(chat_id=CHAT_ID, text=msg, parse_mode="Markdown")
    log.info(f"Message envoyé - {len(picks)} picks.")

async def main():
    log.info("Bot démarré.")

    # Envoi immédiat au démarrage pour tester
    await send_picks()

    # Planification quotidienne
    scheduler = AsyncIOScheduler()
    scheduler.add_job(send_picks, "cron", hour=SEND_HOUR, minute=SEND_MINUTE)
    scheduler.start()
    log.info(f"Envoi planifié chaque jour à {SEND_HOUR:02d}:{SEND_MINUTE:02d} UTC

```

```
# Maintenir le process actif
while True:
    await asyncio.sleep(3600)

if __name__ == "__main__":
    asyncio.run(main())
```